

Actual project details

Taken from the repository as it is. Nothing below is a planned feature.

1. Web application

C. Both React and ASP.NET MVC are implemented.

- React: web-react. React **19.3** (react ^19.3.0), Vite, React Router 7.
- State management: **React Context** (AuthProvider / AuthContext) plus local component state. Not Redux, Zustand, or a global store.
- ASP.NET MVC staff portal: web-app, also implemented. It is a second UI for admin, owner, and agent. It does not replace React.
- React URL if deployed: **Not deployed**. Local only: http://localhost:5173 (npm run dev), preview http://localhost:4173.

Main React pages:

- Public: Home, Login, Register, Hotel detail, Package detail, Forbidden, Not found.
- Traveler: Assistant, My bookings, Itineraries, Profile.
- Hotel owner: Dashboard, Hotels, Rooms, Bookings.
- Travel agent: Dashboard, Packages, Package activities, Transport, Bookings.
- Admin: Dashboard, Approvals, Users, Destinations, Bookings, Reviews, Settings.

2. AI agents

Four specialised agents exist. The fourth is already BookingAgent, not a renamed prompt.

- **Agent 1 — TravelPlanningAgent**. Reads the chat and saved preferences. Extracts destination, dates, duration, travelers, budget (LKR), interests, accommodation and transport. Asks a clarifying question if destination or budget is missing.
- **Agent 2 — RecommendationAgent**. Searches the approved catalog, picks a budget-compliant hotel/room/package/transport combination, and checks live availability.
- **Agent 3 — ItineraryAgent**. Turns that selection into the day-by-day plan, cost breakdown, and reply. Validates the plan against TravelPlanSchema.
- **Agent 4 — BookingAgent**. Builds a priced booking proposal. Creates a booking only after the traveler explicitly confirms. The booking stays Pending until the provider confirms it.

Orchestrator: AgentOrchestrator. Tools: search destinations, hotels, packages, activities, transport, check availability, get preferences, generate itinerary, create booking. OpenAI-compatible tool calling when Ai:ApiKey is set; otherwise the same agents run deterministically.

3. Human approval

YES.

AI plan is validated, then two people must agree before a stay is confirmed:

- **Traveler** must confirm the proposal (reply "confirm", or the confirm id). Nothing is booked before that. The proposal expires (default 15 minutes).
- **Hotel owner** confirms a room booking. **Travel agent** confirms a package booking. An **administrator** can also change booking status.

Separate from the chat: new and edited hotels and packages stay Pending until an **administrator** approves them. Unapproved listings are hidden from search, the AI tools, and booking.

There is no extra "admin approves the AI text" step. Approval is the traveler's confirmation of the proposal, then the provider's confirmation of the Pending booking.

4. AI workflow state

PARTIALLY.

Saved in PostgreSQL:

- AIConversations.Id (conversation id, not a separate workflow-run id)
- User message (the objective) and assistant reply, inside Messages JSON

- Plan, status, pending booking proposal, and created booking id, on that JSON message
- AIRecommendations: item, estimated cost, score, reason, conversation id
- A saved itinerary can store budget, travelers, and the source conversation id

Not stored as their own columns or tables: agent step log, tool-call results, schema validation result, and errors. Those are returned on the live ChatResponse (agents, toolCalls, status) and are not copied into ChatMessageDto.

5. Third-party API / service

- **Third-party service:** OpenAI-compatible chat API. Default `https://api.openai.com/v1`, model `gpt-4o-mini`. `ai:BaseUrl` can point at another compatible host.
- **Purpose:** LLM tool-calling for the travel assistant. If no API key is set, or the provider fails, the deterministic agents still produce a plan from the catalog.
- **Where integrated:** Backend (`OpenAiCompatClient` in Infrastructure). React and Flutter only call `POST /api/ai/chat`.

Unsplash URLs are static image links in seed data, not an API client. Card payment is a simulated gateway inside the API (test card `4000 0000 0000 0002` is declined). Card numbers are not stored. There is no Google Maps, weather, email, SMS, or cloud-storage client.

6. Flutter device feature

Date and time selection. The booking screen uses Flutter `showDatePicker` for check-in and check-out (`create_booking_screen.dart`).

GPS, maps, image picker, QR, push notifications, and file upload are not in the Flutter code.

Flutter state: the provider package (`AuthProvider`, `ChatProvider`, both `ChangeNotifier`).

7. Database

Item	Present
ER diagram file	No image or diagram file. Schema is written in <code>documentation/DATABASE.md</code> and in the EF model.
EF Core migrations	Yes. Six: <code>InitialCreate</code> , <code>AddPackageActivity</code> , <code>AddItineraryPlanFields</code> , <code>AddApprovalStatus</code> , <code>AddHotelAndPackageImages</code> , <code>AddDomainModelV2</code> .
Seed data	Yes. <code>DbSeeder</code> : roles, system settings, optional admin, and demo catalog when demo seeding is on.
Indexes	Yes. Examples: <code>hotel ApprovalStatus</code> ; <code>package (DestinationId, ApprovalStatus)</code> ; <code>unique (Name, Country)</code> on destinations; <code>unique (HotelId, Name)</code> on rooms.
Constraints	Yes. CHECK constraints (price, capacity, dates, rating), foreign keys, unique indexes, and <code>EX_Bookings_Room_NoOverlap</code> (GiST exclusion so one room cannot be double-booked).
Transactions	Yes. <code>UnitOfWork.ExecuteInTransactionAsync</code> . Booking creation locks the room or package row inside a transaction.
CreatedAt / UpdatedAt	Yes, via <code>IAuditable</code> and <code>AuditInterceptor</code> . Not on Identity's extra tables, <code>Roles</code> , or <code>ItineraryItems</code> .

Relationships (from the EF model and DATABASE.md)

- User → Role. User 1:1 `UserProfile`. User 1:1 `TravelPreferences`.
- Hotel owner (user) 1:N Hotels. Hotel 1:N Rooms. Room 1:N `RoomAvailability`.
- Destination 1:N `TravelPackages`. Travel agent (user) creates packages. Package 1:N `PackageActivities`.
- Transportation belongs to an agent, and may point at a package and a destination.
- Booking belongs to a user and to exactly one of Room or `TravelPackage`. Booking 1:N Payments. Booking 1:1 Review (completed bookings).
- Review points at a hotel or a package.
- User 1:N `AIConversations`. Conversation 1:N `AIRecommendations` (hotel, room, package, transport, or destination). User 1:N Itineraries. Itinerary 1:N `ItineraryItems`. Itinerary may point at a destination and a conversation.

8. Testing

- **Backend unit tests:** Yes. `tests/TravelAdvisor.UnitTests` — about 200 [Fact]/[Theory] methods (booking, catalog, auth, users, AI orchestrator, schema, safety, tools, parser, budget).
- **API integration tests:** Yes. `tests/TravelAdvisor.Api.Tests` — about 75 test methods (RBAC, bookings, ownership, security, AI). Also a Postman collection run with Newman.
- **PostgreSQL tests:** Yes. `DatabaseIntegrityTests` uses Testcontainers PostgreSQL (migrations, foreign keys, CHECK, unique, transactions, exclusion constraint).
- **React tests:** Yes. 17 Vitest files (pages, auth, booking panel, API client). 4 Playwright specs: auth, listings, booking, ai-journey.
- **Flutter tests:** Yes. Widget, screen, auth, models, API, chat provider, layout, and app-flow tests, plus `integration_test/traveler_journey_test.dart` and a live API test file.
- **E2E test:** Yes. Playwright traveler flows, and a recorded full traveler journey checked in PostgreSQL (`testing/execution-results`).
- **Agentic AI tests:** Yes. Orchestrator, schema, safety, tool registry, `AiAgentTests`, and `AiEvaluationTests` (scripted cases plus a live-LLM report under `testing/execution-results/phase8-tests/ai-evaluation`).
- **Performance test:** Yes. k6 scripts for catalog search, login, booking, and AI chat (`testing/performance/k6`), with result logs in `testing/execution-results/phase9-nfr`. OWASP ZAP scans also exist.

9. GitHub / CI

- **GitHub repository URL:** <https://github.com/vipooshans/Agentic-AI-Travel-Advisor.git>
- **GitHub Actions implemented:** NO. There is no `.github/workflows` file in the repo.
- **Branches used:** YES. `main`, `dev`, `IT24102892`, `ajay`.
- **Pull requests:** Not verified from this machine (`gh` is not installed). Do not claim PRs without checking GitHub.

Recent commits on dev (newest first):

- `c4c35ea` chore: remove peer dependency flag from `undici-types` in `package-lock.json`
- `30c5c9b` docs: enhance testing documentation for live-LLM evaluation
- `b70eb91` docs: rewrite README, API and architecture docs
- `a0be224` test: add the test-case catalogue and final regression runs
- `5cbdf5e` test(security): OWASP ZAP baseline and OpenAPI scans
- `d5b1855` test(perf): k6 load tests for search, login, booking and AI chat
- `e3369cc` test(e2e): full traveler journey verified in PostgreSQL

No GitHub Actions success screenshot exists, because Actions is not set up.

10. Deployment

- **React URL:** Not deployed. Local `http://localhost:5173`.
- **API URL:** Not deployed. Local `http://localhost:5000` (launch profile) or `http://localhost:5080` (README / React proxy).
- **Swagger URL:** Not deployed. Local `http://localhost:5000/swagger` in Development.
- **PostgreSQL:** Not a hosted database. Docker Compose maps it to `localhost:5433`. Local dev docs also use port 5432.
- **Flutter APK:** Not in the repository. No `.apk` file.
- **AI service/setup:** Optional `Ai:ApiKey`, `Ai:BaseUr1` (default OpenAI), `Ai:Model` (`gpt-4o-mini`). Docker uses `AI_API_KEY`. Without a key the deterministic planner still runs.

11. Four students

Not recorded in the repository.

The only identity evidence in git is:

- GitHub account: `vipooshans`
- Branch name: `IT24102892` (looks like a student ID; no name is stored with it)
- Branch name: `ajay` (no student ID stored with it)

Student 1–4 names, IDs, and owned components are not in the code, README, or commit authors (local history shows only `vipooshans`). Do not fill the four names from guesswork.

12. ADRs

NO. There is no ADR folder.

- React state management: no ADR. Actual choice: React Context.
- Flutter state management: no ADR. Actual choice: provider.
- Agentic AI orchestration: no ADR. Actual choice: custom AgentOrchestrator plus four agents and a tool registry. Not Semantic Kernel, LangGraph, or AutoGen.
- Database / workflow state: no ADR. Actual choice: PostgreSQL 16, EF Core 8, conversation JSON plus AIRecommendations. Not a full workflow-state table.
- Cloud / deployment: no ADR. Actual choice: Docker Compose for API, MVC portal, and PostgreSQL. React, Flutter, and a public host are not deployed.